

Buổi 10 — Làm sạch và dữ liệu thiếu

1. Mục tiêu

Sau buổi này bạn:

- Phân biệt **thiếu mốc** (không có dòng) với **thiếu giá trị** (có dòng, ô rỗng) — và biết vì sao `isna()` không thấy loại thứ nhất.
- Gọi tên cơ chế thiếu **MCAR / MAR / MNAR** và chứng minh bằng số rằng MNAR làm lệch kết quả dù điển kiểu gì.
- Phát hiện bốn thứ không phải số đo: **mã trá hình, trần cảm biến, độ phân giải thô, cảm biến đứng yên**.
- Đọc **cờ chất lượng** của nguồn dữ liệu chuẩn (GHCNh) thay vì tự đoán.
- So sánh 7 cách điển bằng **che nhân tạo**, và thấy thứ hạng **đảo** giữa lỗ ngắn và lỗ dài.
- Viết pipeline làm sạch **có giới hạn, có cờ truy vết**, kèm test tự động — trong đó có test chống rò rỉ tương lai.

2. Nhắc lại buổi trước

- **Đặc trưng chuỗi** (buổi 9): chuỗi hằng làm `kpss` ném lỗi; `ty_le_0` và `doan_phang` là dấu hiệu chuỗi lạ. Hôm nay ta đi sâu vào **vì sao** chuỗi lại có những dấu hiệu đó.
- **Thước đo quyết định kết luận**: cùng một dự báo, SMAPE nói khác MASE. Hôm nay: cùng một bộ phương pháp điển, **kiểu che** quyết định phương pháp nào "tốt nhất".
- **Backtest trung thực**: mọi đánh giá phải có đáp án thật để đối chiếu. Với việc điển dữ liệu, "đáp án thật" có được bằng cách **che những ô đang có giá trị**.

3. Trạng thái đầu buổi

Sau `cd lab && make up`:

Hạng mục	Trạng thái
Dữ liệu 1	<code>ghcnh-noi-bai-2024/GHCNh_VMI0000VVNB_2024.parquet</code> — 17.319 dòng mốc 30 phút, cả năm 2024, 329 cột, sha256 <code>1c67048928d4</code>
Dữ liệu 2	<code>uci-beijing-air/</code> — 12 trạm × 35.064 giờ PM2.5, 2013-03 → 2017-02
Dữ liệu 3	<code>open-meteo-ha-noi-2023-2024.csv</code> — 17.547 giờ , dùng làm "trạm hàng xóm"
Nguồn/giấy phép	NOAA GHCNh (CC0-1.0, trạm Việt Nam thật); UCI Beijing (CC BY 4.0); Open-Meteo (CC BY 4.0)
Môi trường	Python 3.12.14; numpy 2.5.3, pandas 3.0.5, scipy 1.18.1, statsmodels 0.15.0, pyarrow
<code>code/</code> <code>lam_sach.py</code>	<code>doc_noi_bai</code> , <code>cot_rong</code> , <code>luoi_day_du</code> , <code>thong_ke_thieu</code> , <code>do_phan_giai</code> , <code>doan_mac_ket</code> , <code>doan_tra_hinh</code> , <code>co_nghi_ngo</code> , <code>bao_cao_chat_luong</code> , 7 hàm <code>dien_*</code> , <code>che_diem</code> , <code>che_khoi</code> , <code>so_sanh_dien</code> , <code>lam_sach</code> , <code>mo_phong_mnar</code>
Đang cố tình sai	<code>doc_noi_bai</code> giữ nguyên kiểu dữ liệu của parquet; <code>so_sanh_dien</code> chỉ che ngẫu nhiên từng điểm; <code>lam_sach</code> điển mọi lỗ bằng nội suy hai phía

Hạng mục	Trạng thái
Triệu chứng	Báo cáo ghi độ ẩm min = 100, max = 94 ; bảng xếp hạng kết luận "nội suy tuyến tính luôn tốt nhất"; sau làm sạch "còn thiếu: 0" — sạch một cách đáng ngờ
make check lúc này	ĐỎ: 7/12 test qua, 5 test hỏng

4. Lý thuyết

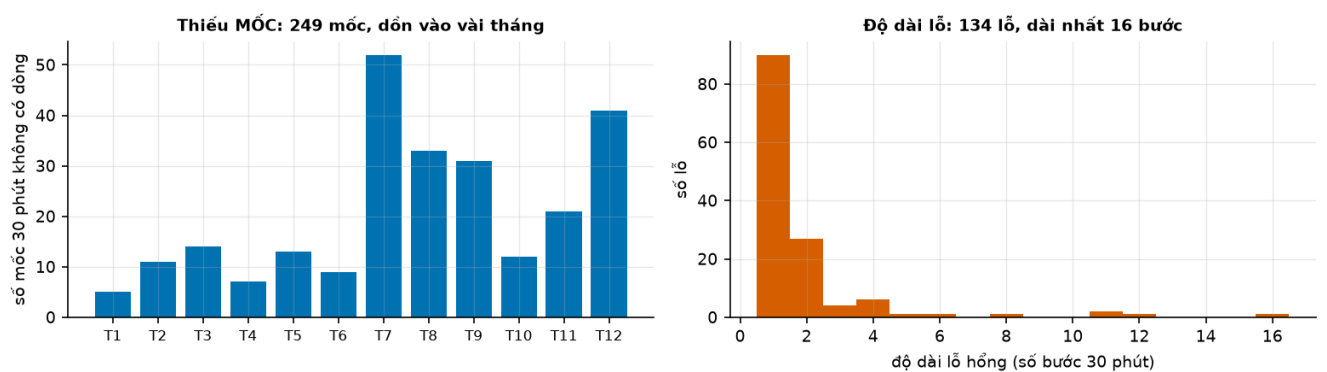
4.1 Hai loại thiếu, và loại thứ ba

```
tk = thông_ke_thieu(tho)
# {'so_moc_ky_vong': 17568, 'so_moc_co': 17319, 'thieu_moc': 249, 'moc_trung': 0, 'thieu_gia_tri': 2}
```

Năm 2024 có 17.568 mốc 30 phút. Tập có 17.319 dòng và **chỉ 2 ô rỗng**. Nếu bạn chỉ chạy `df.isna().sum()` thì kết luận là "dữ liệu gần như hoàn hảo" — trong khi **249 mốc không hề tồn tại**. Bước đầu tiên của mọi pipeline chuỗi thời gian:

```
luoi = pd.date_range(bang.index.min(), bang.index.max(), freq="30min")
day_du = bang.reindex(luoi) # thiếu mốc → hiện ra thành dòng NaN
```

Hai loại thiếu khác nhau: không có DÒNG (trái) khác với có dòng mà ô RỖNG — chỉ `isna()` sẽ không thấy loại thứ nhất



Đọc hình. Trái: 249 mốc thiếu **dồn vào vài tháng** chứ không rải đều (tháng nhiều nhất 52 mốc) — dấu hiệu của sự cố hệ thống, không phải nhiễu ngẫu nhiên. Phải: 134 lỗ hỏng, **67,2% chỉ dài 1 bước**, lỗ dài nhất 16 bước (8 giờ). Phân bố này quyết định bạn được phép điền tới đâu.

Loại thứ ba: số 0 trá hình. Doanh số = 0 vì cửa hàng đóng cửa, hay vì hết hàng? Ô nhiễm = 0 hay cảm biến mất tín hiệu? Đây là **censored demand**: ngày hết hàng, nhu cầu thật cao hơn doanh số ghi nhận. Mô hình học từ số 0 đó sẽ dự báo thấp mãi mãi, và càng dự báo thấp thì càng hết hàng — một vòng lặp tự củng cố. Cách xử lý: đánh dấu ngày hết hàng là **thiếu**, không phải bằng 0.

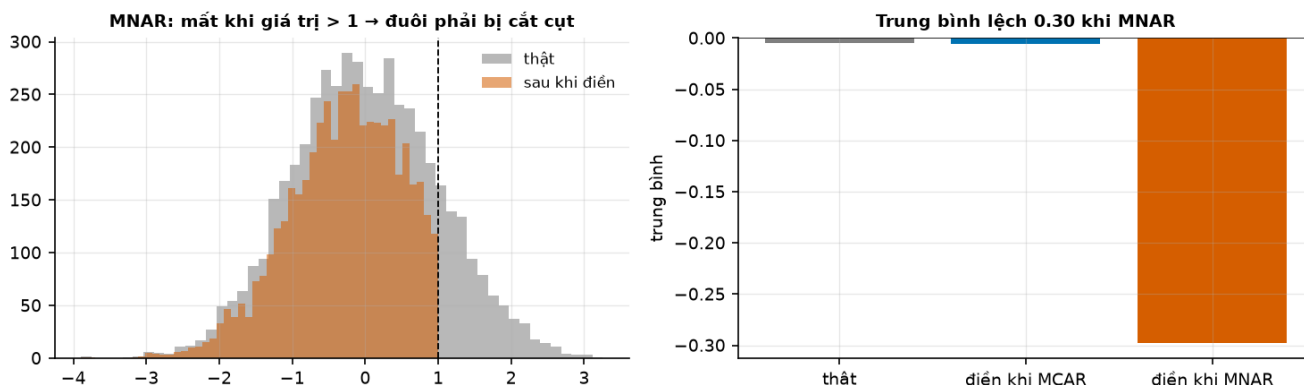
4.2 Cơ chế thiếu: MCAR, MAR, MNAR

Theo Rubin (1976):

- **MCAR** (missing completely at random): xác suất thiếu không phụ thuộc gì. Mất mạng vài phút. Điền thoải mái.
- **MAR** (missing at random): xác suất thiếu phụ thuộc vào **dữ liệu quan sát được** (ví dụ trạm hay hỏng vào mùa mưa, mà mùa thì ta biết). Điền được, nếu mô hình điền có biến giải thích đó.

- **MNAR** (missing not at random): xác suất thiếu phụ thuộc **chính giá trị bị thiếu**. Cảm biến quá tải khi ô nhiễm cực cao và tắt. **Không** điền được từ dữ liệu quan sát được, dù mô hình mạnh đến đâu.

Cơ chế thiếu quyết định điền được hay không: MCAR diễn vô tư, MNAR thì diễn kiểu gì cũng lệch

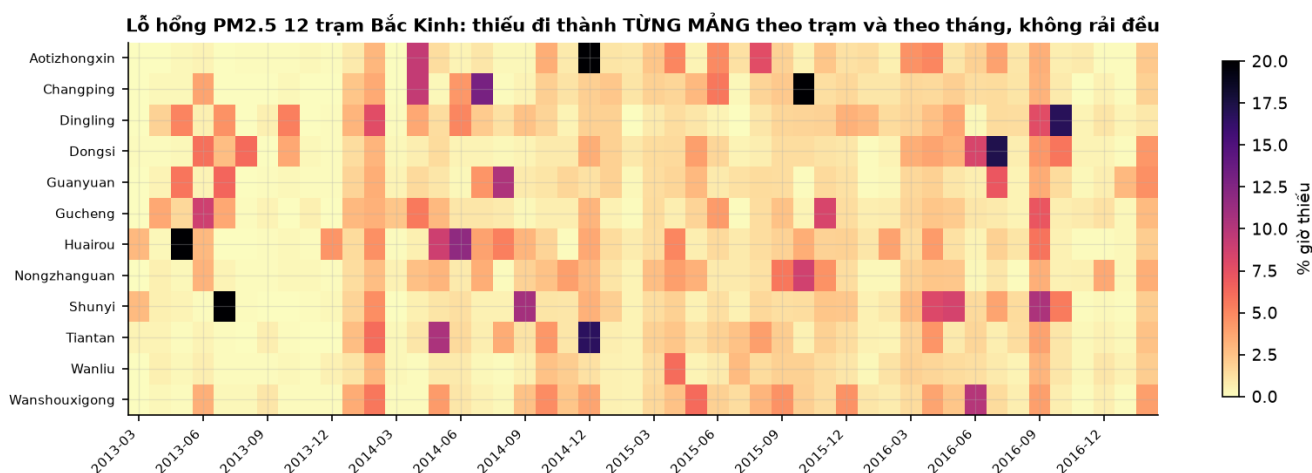


Độc hình. Mô phỏng: cảm biến tắt khi giá trị > 1 (mất 16,2% số điểm). Trái: đuôi phải của phân phối bị **cắt cụt** sau khi điền. Phải: trung bình thật -0,005; điền khi MCAR ra -0,006 (không lệch); điền khi MNAR ra **-0,298** — lệch gấp khoảng 50 lần.

Kiểm MNAR trên dữ liệu thật. Không đoán, hãy đo: khi trạm A mất dữ liệu, mức PM2.5 của các trạm còn lại cao hay thấp hơn bình thường?

```
bang_chung_mnar(bk, "Dongsi")
# {'tỷ lệ thiếu %': 2.14, 'PM2.5 hàng xóm khi trạm thiếu': 76.8,
#  'PM2.5 hàng xóm khi trạm có': 79.2, 'chênh lệch %': -3.0}
```

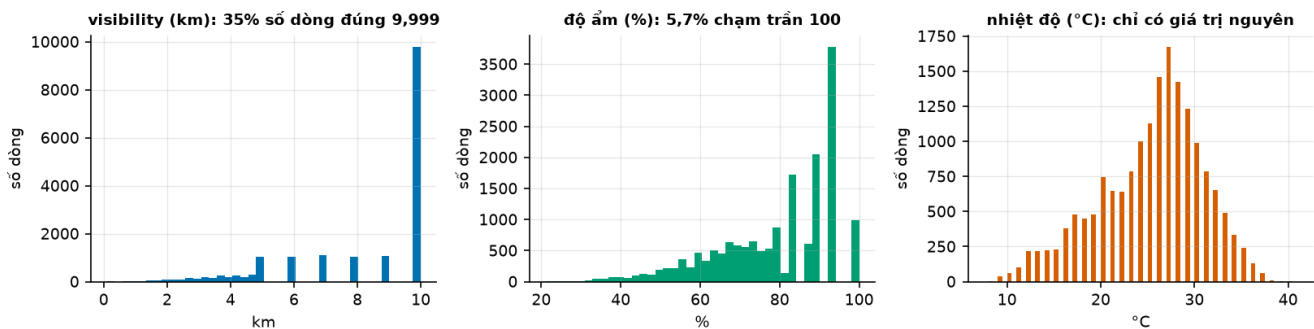
Kết quả **-3,0%**: không có bằng chứng MNAR ở bộ Bắc Kinh. Đây là kết quả thật, và cũng là bài học: **phải kiểm, đừng giả định theo cả hai chiều**. (Ba trạm được thử đều cho chênh lệch âm, từ -3,0% đến -14,1%.)



Độc hình. 12 trạm Bắc Kinh, tỷ lệ thiếu theo tháng. Thiếu trung bình chỉ 2,08%, nhưng phân bố thành **mảng**: 13 ô (trạm × tháng) có trên 10% thiếu, ô tẻ nhất 48,0%. Một con số "2% thiếu" trong báo cáo hoàn toàn che mất chuyện một trạm chết nửa tháng.

4.3 Bốn thứ trông giống số đo mà không phải

Ba thứ không phải 'số đo': mã trá hình, trần cảm biến, và độ phân giải thô — histogram lộ ra cả ba



Đọc hình. Ba histogram, ba vấn đề khác nhau:

1. **Mã trá hình.** `visibility = 9.999 km` chiếm **35,0%** số dòng. Đó không phải số đo: METAR quy định "visibility is 10 km and above ... shall be indicated as 9999" (đơn vị mét), GHCNh đổi sang km thành 9.999. Tính trung bình tầm nhìn với con số này là sai.
2. **Trần cảm biến.** `relative_humidity = 100%` ở **5,68%** số dòng — cảm biến độ ẩm bão hoà. Giá trị thật có thể là 100% hoặc 103%, ta không biết; đây là dữ liệu **bị kiểm duyệt phía trên**.
3. **Độ phân giải thô.** Nhiệt độ chỉ có **33 giá trị khác nhau**, bước nhỏ nhất **1,0 °C**, 100% là số nguyên.

Điều thứ ba dẫn tới cái bẫy quan trọng nhất của mục này. Chuỗi nhiệt độ có đoạn **67 bước liên tiếp bằng đúng 26,0 °C** (33,5 giờ, từ 2024-06-08 16:30). Cảm biến chết? Có thể. Nhưng nếu hạ ngưỡng xuống 24 bước (12 giờ) thì có tới **24 đoạn, 719 điểm (4,15%)** — quá nhiều để tất cả đều hỏng. Lý do: với độ phân giải 1 °C, một đêm nhiệt độ thật đổi 0,4 °C vẫn hiện ra thành chuỗi giá trị lặp.

Quy tắc: đo độ phân giải **trước**, rồi mới đặt ngưỡng "đứng yên". Trong bài dùng 36 bước (18 giờ) → còn 2 đoạn, 123 điểm.

Cờ chất lượng — đọc tài liệu, đừng đoán. GHCNh ghi rõ (Table 3): `1` = "Passed all quality control checks", `2` = "Suspect", `3` = "Erroneous", `4` = "Passed gross limits check, data originate from an NCEI data source", `6 / 7` = Suspect/Erroneous từ nguồn NCEI. Với độ ẩm suy ra từ biến khác: `o` = "Out of range", `f` = "Suspect or Error flags for 1 or more of the input measurements".

Rất nhiều người nhầm mã `4` là "lỗi". Không phải — nó chỉ nghĩa là dữ liệu mới qua kiểm tra giới hạn thô. Nhóm đáng ngờ đúng là `{2, 3, 6, 7, o, f}`: ở trạm này có **7 dòng** nhiệt độ mã `2` và 7 dòng độ ẩm cờ `f`.

Cái bẫy kiểu dữ liệu. Trong parquet gốc, `relative_humidity` và `visibility` lưu dạng **chuỗi**. Không có exception nào, chỉ có kết quả sai:

```
tho["relative_humidity"].min(), tho["relative_humidity"].max() # ('100', '94') ← so theo thứ tự chữ cái
```

Vì vậy `doc_noi_bai` phải `pd.to_numeric(..., errors="coerce")` cho **mọi** cột đo. Bài học chung: sau khi đọc dữ liệu, việc đầu tiên là in `dtypes` và một bảng min/max — hai lệnh rẽ tiền bắt được cả lớp lỗi.

4.4 Ba lỗi "hệ thống" khó thấy: mốc trùng, đổi đơn vị, đổi múi giờ

Ba lỗi này không tạo ra NaN nào, nên mọi kiểm tra dựa trên `isna()` đều bỏ sót.

Mốc trùng. Hai dòng cùng một thời điểm. Nguyên nhân thường gặp: ghép hai nguồn, hoặc đồng hồ lùi lại một giờ khi đổi giờ mùa hè.

```
bang.index.duplicated().sum()      # trạm Nội Bài: 0 – nhưng phải kiểm, không được giả định
bang.groupby(level=0).size().max() # nếu > 1: quyết định giữ dòng nào, và GHI LẠI quyết định đó
```

`reindex` lên lưới đều sẽ **ném lỗi** nếu chỉ số có giá trị trùng — đó là lý do bước kiểm này phải đứng trước.

Đổi đơn vị giữa chừng. Nồng độ khí ghi bằng $\mu\text{g}/\text{m}^3$ hay ppb tùy nguồn và tùy giai đoạn; nhiệt độ có nguồn ghi $\times 10$ ($260 = 26,0^\circ\text{C}$). Dấu hiệu: **mức trung bình nhảy bậc đúng vào một mốc tròn** (đầu năm, đầu tháng, ngày thay thiết bị) trong khi phương sai tương đối không đổi. Cách kiểm rẻ: vẽ trung bình theo tháng, và so dải giá trị của từng năm:

```
y.groupby(y.index.year).agg(["min", "max", "mean"]) # một năm lệch hẳn thang = nghi đổi đơn vị
```

Dải nhiệt độ Nội Bài là $8\text{--}40^\circ\text{C}$ — hợp lý cho Hà Nội, nên không có đổi đơn vị; thấy $\text{max} = 400$ thì đó là $\times 10$.

Đổi múi giờ. Nguy hiểm nhất vì chuỗi vẫn "liền mạch". Dấu hiệu: **nhịp ngày bị dịch pha** giữa hai giai đoạn — nhiệt độ cao nhất chuyển từ 13 h sang 6 h. Cách kiểm: tính trung bình theo giờ trong ngày cho từng nửa dữ liệu rồi so đỉnh.

```
dinh = y.groupby([y.index.year, y.index.hour]).mean().groupby(level=0).idxmax()
```

GHCNh ghi thời gian theo **UTC**, Open-Meteo trong tệp của buổi này cũng UTC (`timezone: GMT` ở dòng đầu) — nên ghép được trực tiếp. Nếu một trong hai nguồn dùng giờ địa phương mà ta không để ý, tương quan sẽ tụt từ 0,976 xuống gần 0 chỉ vì lệch 7 giờ. **Quy ước cho cả khoá:** lưu và tính toán bằng UTC, chỉ đổi sang giờ địa phương khi hiển thị hoặc khi tạo feature lịch (buổi 13).

4.5 Bẫy cách điền

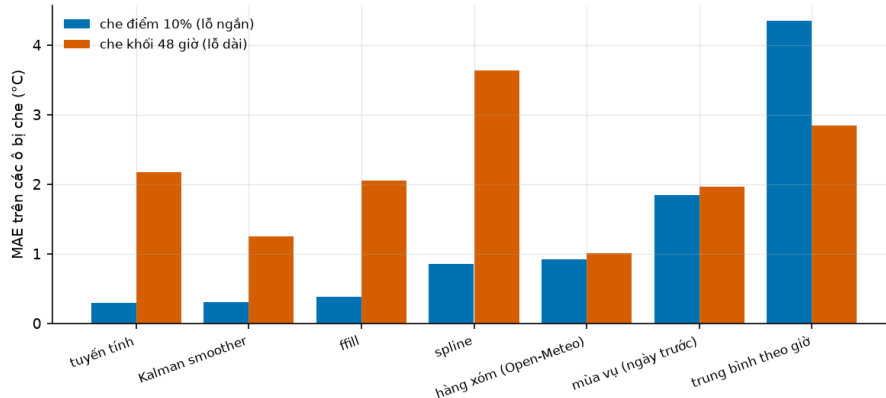
Cách	Ý tưởng	Dùng khi	KHÔNG dùng khi
<code>ffill</code>	giữ giá trị gần nhất	lỗ rất ngắn; dữ liệu bậc thang (tồn kho, giá)	lỗ dài — tạo đoạn phẳng giả
tuyến tính	nối hai đầu lỗ	lỗ ngắn, chuỗi mượt	lỗ dài; và dùng tương lai
spline	nối mượt bậc 3	tín hiệu mượt, lỗ ngắn	lỗ dài — vọt lỗ mạnh (MAE 3,636)
mùa vụ (ngày trước)	lấy giá trị cùng giờ hôm trước	nhịp ngày/tuần mạnh, lỗ dài	chuỗi không có mùa vụ
trung bình theo giờ	khí hậu học	thiếu rất nhiều, cần giá trị "hợp lý"	khi cần chi tiết ngày cụ thể
Kalman smoother	mô hình không gian trạng thái	mọi độ dài lỗ; có mô hình cấu trúc	cần nhân quả (smoother dùng cả chuỗi)
trạm hàng xóm	hồi quy theo chuỗi tương quan	có nguồn tương quan cao (ở đây $r = 0,976$)	không có hàng xóm, hoặc hàng xóm cùng hỏng

Kalman smoother lấy từ `smoother_results.smoothed_forecasts[0]` — **không** dùng `fittedvalues` (đó là dự báo một bước, không phải giá trị đã làm trơn).

4.6 Đánh giá: che nhân tạo, và phải che hai kiểu

Muốn biết cách điền nào tốt, hãy **che những ô đang có giá trị** rồi so với đáp án. Nhưng che thế nào?

Thứ hạng ĐẢO: nội suy tuyến tính hạng 1/7 với lỗ ngắn (0,29) nhưng tụt xuống 5/7 với lỗ 48 giờ (2,17); trạm hàng xóm đi ngược lại, từ 5/7 lên 1/7

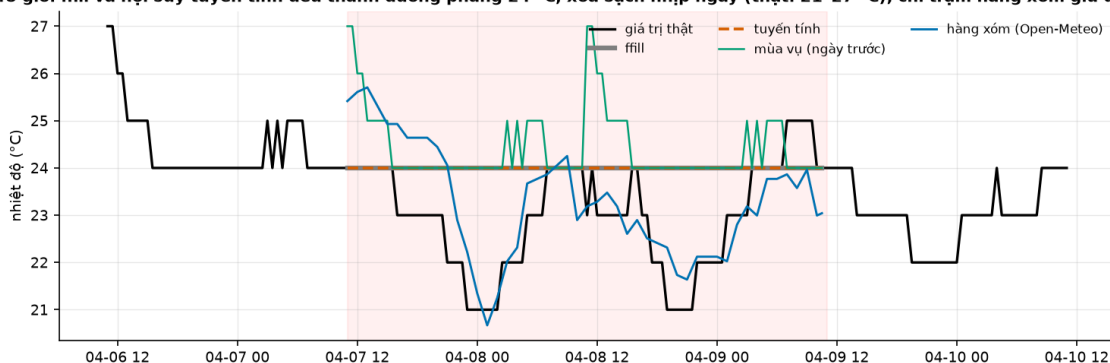


Độc hình. Cùng 7 phương pháp, cùng một chuỗi, hai kiểu che:

Cách điền	Che điểm 10% (lỗ ngắn)	Che khối 48 giờ (lỗ dài)
tuyến tính	0,291 — hạng 1	2,171 — hạng 5
Kalman smoother	0,308	1,256
ffill	0,378	2,052
spline	0,857	3,636
hàng xóm (Open-Meteo)	0,922 — hạng 5	1,006 — hạng 1
mùa vụ (ngày trước)	1,845	1,965
trung bình theo giờ	4,350	2,839

Nội suy tuyến tính từ hạng 1 tụt xuống hạng 5; trạm hàng xóm đi ngược lại. Lý do rõ khi nhìn một lỗ cụ thể:

Một lỗ 48 giờ: ffill và nội suy tuyến tính đều thành đường phẳng 24 °C, xóa sạch nhịp ngày (thật: 21-27 °C); chỉ trạm hàng xóm giữ được hình dạng



Độc hình. Trong 48 giờ bị che, nhiệt độ thật dao động 21-27 °C theo nhịp ngày. `ffill` cho một đường phẳng 24 °C; nội suy tuyến tính cũng gần như vậy (hai đầu lỗ tình cờ bằng nhau). Cả hai **xóa sạch nhịp ngày** — và nếu bạn huấn luyện mô hình trên chuỗi đó, mô hình sẽ học rằng có những giai đoạn nhiệt độ đứng yên hai ngày liền. Chỉ trạm hàng xóm giữ được hình dạng.

Quy tắc đánh giá: kiểu che phải **giống cách dữ liệu thật bị mất**. Ở trạm Nội Bài, 67% lỗ dài 1 bước nhưng cũng có lỗ 8 giờ, nên phải báo cáo cả hai. Nếu chỉ che điểm, bạn sẽ chọn nội suy tuyến tính và nó sẽ hỏng đúng lúc quan trọng nhất.

4.7 Điền dữ liệu là một dạng rò rỉ

`interpolate(limit_direction="both")` dùng giá trị **sau** thời điểm đang điền. Nếu điền trước khi chia train/test, thông tin từ test rò sang train. Bài kiểm tự động (buổi 12 sẽ dựng kỹ) cắt dữ liệu tại một mốc và so kết quả ở phần trước:

```
kq = ro_ri.kiem_ro_ri(ham_lam_sach, du_lieu, cac_moc_cat=[moc_trong_lo], h=None, cot_id=None)
kq.khang_dinh_sach()
```

Mẹo quan trọng: **mốc cắt phải nằm trong một lỗ hổng**. Cắt ở chỗ dữ liệu đầy đủ thì phương pháp dùng tương lai vẫn cho kết quả y hệt, và bài kiểm im lặng.

Thứ tự đúng: **chia tập → điền trong từng tập bằng phương pháp nhân quả** (hoặc điền test bằng tham số học từ train). `ffill` và "mùa vụ ngày trước" nhân quả; tuyến tính, spline, Kalman smoother, trung bình toàn chuỗi thì không.

4.8 Lỗ dài: đừng điền, hãy đánh dấu

Điền 48 giờ là **bị dữ liệu**. Pipeline đúng có ba cột đầu ra:

```
sach = lam_sach(tho, gioi_han=6, cach=dien_mua_vu, hang_xom=hx)
# cột: temperature | da_dien | nghi_ngo | lo_dai_bo_trong
bao_cao_lam_sach(sach)
# {'số mốc': 17568, 'còn thiếu': 181, 'đã điền': 198,
#  'bị loại vì nghi ngờ': 130, 'ô trong lỗ dài (để trống có chủ ý)': 181}
```

Cột `da_dien` cho phép: (a) huấn luyện có trọng số thấp hơn ở ô đã điền, (b) loại các ô đã điền khỏi tập chấm, (c) trả lời được câu hỏi "số này từ đâu ra" sáu tháng sau. Dữ liệu sạch mà **không truy vết được** thì không dùng được cho báo cáo nghiêm túc.

Deep learning thì sao? SAITS/BRITS (qua PyPOTS) và benchmark TSI-Bench cho thấy lợi thế xuất hiện khi có **nhiều chuỗi tương quan** và tỷ lệ thiếu cao. Với một trạm và 2% thiếu, hồi quy theo trạm hàng xóm đã đạt 1,006 °C — không cần torch. Biết để chọn, không cần cài.

4.9 Thứ tự tám bước — làm đúng thứ tự mới đúng kết quả

Làm sạch sai thứ tự thì mỗi bước sau che mất bằng chứng của bước trước. Thứ tự dùng trong `lam_sach`:

1. **Đọc và ép kiểu** — `dtypes` + bảng min/max trước khi tin bất cứ con số nào.
2. **Lưới thời gian đều** — `reindex`; kiểm mốc trùng và múi giờ. (Một mốc lặp hai lần thường là do đổi giờ hoặc ghép hai nguồn; ở trạm này là 0.)
3. **Bỏ cột rỗng** — 30/64 cột ở đây không có một giá trị nào.
4. **Tra cờ chất lượng của nguồn** — rõ nhất và chính xác nhất, vì người tạo dữ liệu biết rõ hơn ta.
5. **Mã trá hình và trần cảm biến** → NaN + cờ `nghi_ngo`; đừng xóa dòng, chỉ xóa **giá trị**.
6. **Độ phân giải → ngưỡng đứng yên** → loại đoạn cảm biến chết.
7. **Đo lại phân bố độ dài lỗ**, rồi chọn `gioi_han` điền và phương pháp điền (nhân quả).
8. **Xuất dữ liệu + cờ + báo cáo** trong cùng một lần chạy, có seed.

Bước 5 hay bị làm thành `dropna()` cả dòng. Đừng: một dòng có `visibility = 9.999` vẫn có nhiệt độ hợp lệ; xóa cả dòng là vớt 35% dữ liệu nhiệt độ vì vấn đề của cột khác. Bước 7 phải làm **sau** bước 5-6, vì biến giá trị nghi ngờ thành NaN có thể nối các lỗ ngắn thành lỗ dài: ở đây 130 giá trị bị loại, kết quả cuối là 198 ô được điền và 181 ô cố tình để trống.

Một con số để nhớ: tương quan giờ giữa nhiệt độ trạm Nội Bài và Open-Meteo Hà Nội là $r = 0,976$ ($n = 8.716$, lệch trung bình $-0,08\text{ }^{\circ}\text{C}$). Chính vì tương quan cao như vậy mà hồi quy theo "hàng xóm" mới thắng ở chỗ dài. Trước khi dùng nguồn phụ để điền, **hãy đo tương quan trên phần hai bên cùng có dữ liệu** — nếu r thấp, bạn đang bơm nhiễu của nguồn khác vào chuỗi của mình.

5. Lab từng bước

Bước 1 — Nhìn lỗi hồng trước khi làm gì khác

```
cd lab && make up
env -u VIRTUAL_ENV uv run --no-sync --project 00-nen python ../code/lam_sach.py
make check # 5/12 đỏ
```

Vẽ heatmap thiếu cho 12 trạm Bắc Kinh. Trả lời: trạm nào không dùng được cho giai đoạn nào?

Bước 2 — Báo cáo chất lượng cho trạm Nội Bài

Sửa `doc_noi_bai` để ép kiểu số. Chạy `bao_cao_chat_luong` và `do_phan_giai`; giải thích ba con số: 35,0% / 5,68% / 1,0 °C. Đặt ngưỡng "đứng yên" và biện luận tại sao.

Bước 3 — Che hai kiểu, so 7 cách điền

Sửa `so_sanh_dien` để chạy cả `che_diem` và `che_khoi`. Lập bảng hai cột MAE và chỉ ra cặp phương pháp **đổi chỗ** cho nhau.

Bước 4 — Pipeline có giới hạn và có cờ

Sửa `lam_sach`: chỉ điền lỗ $\leq$ `gioi_han`, dùng phương pháp **nhân quả**, sinh đủ ba cờ. Chạy lại bài kiểm rờ rĩ với mốc cắt nằm trong lỗ.

```
make check # 12/12 xanh
```

6. Lỗi thường gặp & cách chẩn đoán

Triệu chứng	Nguyên nhân	Chẩn đoán	Sửa
<code>isna()</code> báo dữ liệu gần như đủ, mà mô hình vẫn lạ	thiếu mốc , không phải thiếu giá trị	<code>so len(df)</code> với <code>len(date_range(...))</code>	<code>reindex</code> lên lưới đều trước mọi việc khác
<code>min</code> lớn hơn <code>max</code>	cột số lưu dạng chuỗi	<code>in df.dtypes</code>	<code>pd.to_numeric(..., errors="coerce")</code>
Một giá trị chiếm 35% số dòng	mã trá hình (sentinel)	<code>value_counts().head()</code>	tra tài liệu nguồn; đổi thành NaN + cờ
Rất nhiều "cảm biến đứng yên"	độ phân giải thô	<code>do_phan_giai</code>	đặt ngưỡng theo độ phân giải
Loại quá nhiều dòng theo cờ chất lượng	hiểu sai bảng mã (vd coi <code>4</code> là lỗi)	đọc Table 3 của nguồn	chỉ loại <code>{2, 3, 6, 7, o, f}</code>

Triệu chứng	Nguyên nhân	Chẩn đoán	Sửa
Trung bình lệch sau khi điền	thiếu MNAR	so giá trị hàng xóm khi thiếu / khi có	không điền; mô hình hoá cơ chế thiếu; báo cáo giới hạn
Chuỗi có đoạn phẳng dài bất thường	điền lỗ dài bằng ffill/ tuyến tính	vẽ chuỗi quanh lỗ	đặt <code>gioi_han</code> ; để lỗ dài là NaN
Backtest tốt bất thường sau khi làm sạch	điền trước khi chia tập / dùng tương lai	<code>kiem_ro_ri</code> với mốc cắt trong lỗ	chia tập trước; dùng cách điền nhân quả
"Phương pháp X luôn tốt nhất"	chỉ che một kiểu	che thêm kiểu khối	báo cáo cả hai kiểu che
Mô hình dự báo thấp mãi cho hàng hay hết	censored demand: 0 do hết hàng	so ngày tồn kho = 0	đánh dấu ngày hết hàng là thiếu
Cột toàn NaN nhưng vẫn vào pipeline	nguồn có cột nhưng không có dữ liệu cho trạm này	<code>isna().all()</code>	bỏ sớm (30/64 cột ở đây)
Không giải thích được một giá trị 6 tháng sau	không có cờ truy vết	tìm cột <code>da_dien</code>	luôn xuất cờ cùng dữ liệu sạch

7. Bài tập về nhà

- Che theo đúng phân bố thật.** Ước lượng phân bố độ dài lỗ của trạm Nội Bài (67% lỗ 1 bước...), rồi sinh mặt nạ che theo đúng phân bố đó. Bảng xếp hạng thay đổi thế nào so với hai kiểu che trong bài?
- MAR có cứu được không.** Tạo thiếu MAR trên dữ liệu Bắc Kinh: xoá PM2.5 khi tốc độ gió cao. So sai số điền khi mô hình điền **có** và **không** dùng cột gió. Rút ra kết luận về "biến giải thích cơ chế thiếu".
- Giá của việc điền sai.** Huấn luyện một dự báo 1 giờ trên chuỗi đã điền bằng (a) nội suy tuyến tính mọi lỗ, (b) chỉ điền lỗ ≤ 3 giờ và bỏ phần còn lại. So MAE trên tập kiểm **không** có ô nào bị điền.

8. Tiêu chí "Xong khi"

- [] `make_check` xanh 12/12.
- [] Nộp báo cáo chất lượng dữ liệu tự sinh cho trạm Nội Bài: thiếu mốc, cột rỗng, mã trá hình, độ phân giải, cờ chất lượng.
- [] Bảng so sánh 7 cách điền trên **hai** kiểu che, kèm một câu giải thích vì sao thứ hạng đảo.
- [] Pipeline có `da_dien`, `ngghi_ngo`, `lo_dai_bo_trong`; lỗ > 3 giờ vẫn là NaN.
- [] Bài kiểm rò rỉ xanh **với mốc cắt nằm trong lỗ hổng**.
- [] Trả lời bằng số: nếu cảm biến của bạn tắt khi giá trị cao, việc điền làm trung bình lệch bao nhiêu?

9. Đọc thêm

- Rubin, D.B. (1976). Inference and missing data. *Biometrika* 63(3), 581–592.
- van Buuren, S. *Flexible Imputation of Missing Data*, 2nd ed. — <https://stefvanbuuren.name/fimd/>

- NOAA GHCNv documentation — https://www.ncei.noaa.gov/oa/global-historical-climatology-network/hourly/doc/ghcnv_DOCUMENTATION.pdf
- Du, W. et al. PyPOTS / TSI-Bench — <https://github.com/WenjieDu/PyPOTS>, arXiv:2406.12747
- pandas `interpolate`, `reindex`, `asfreq` — https://pandas.pydata.org/docs/user_guide/missing_data.html
- statsmodels state space: `UnobservedComponents`, `smoother_results` — <https://www.statsmodels.org/stable/statespace.html>
- METAR/ICAO visibility 9999 — <https://skybrary.aero/articles/meteorological-aerodrome-report-metar>
- Chen, S.X. (2019). Beijing Multi-Site Air-Quality Data. UCI ML Repository. <https://doi.org/10.24432/C5RK5G>